
Printing badges at an FE event

The print partner interface

We announce every badge that we must print, and your system does the rest.

Spec version 1

17 September 2026

Forum Europe — event badge printing

Contents

1. Words we use	3
2. Who does what	4
3. The steps in short	4
4. Connecting	5
5. The announcement	5
6. Collecting a badge	6
7. The waiting list	12
8. Timing, cancellation and repeats	12
8a. Never print a badge twice	13
9. Status codes	15
10. Your data duties	16
11. Testing	16
12. Troubleshooting	17
13. What is not available	17
Changes	17
A. Getting started (the kit's README)	17

The model in one sentence: we announce every badge that we must print, and your system does the rest.

At an FE event, our check-in desks (iPads) ask our print hub for a badge. The hub does not print the badge. The hub keeps the badge. The hub sends an announcement on the event network. The hub then waits for your system to collect the badge over HTTP. You print the badge in the way that you want.

This document gives you all that you need. The developer kit contains these files:

File	What it is
<code>print-partner-spec.md</code>	This document.
<code>print-partner-client.py</code>	An example client. It listens, it collects, and it saves each badge as a PNG file and a JSON file. You put your printing in one marked function.
<code>fake-partner-hub.py</code>	A stand-in hub for your own computer. It makes badges with false data. Use it to build and to test without us.
<code>samples/</code>	One real announcement, one real badge (a test badge, with no real person) and the picture of that badge.

Both scripts need Python 3.10 or a later version. They need nothing more. They run on Windows and on macOS.

1. Words we use

We use one word for one thing. This table gives each word.

Word	What it means
announcement	The small UDP datagram that the hub sends on UDP port <code>8632</code> to tell you that a badge is ready. Earlier versions of this document also called it the datagram or the doorbell.
badge	The thing that you print, and also the JSON data for it.
job id	The hub's id for one offer of a badge. It is the <code>id</code> field.
job key	The desk's id for the badge. It is the <code>jobKey</code> field. It stays the same across a desk retry.
collector	Your system, which collects badges. It gives its name in the <code>X-Print-Collector</code> header.
collect	To make a <code>GET</code> request for a badge. The first <code>GET</code> takes the badge.
waiting list	The list of badges that nobody collected. You get it with <code>GET /v1/prints</code> .
cancelled	The state of a badge that the hub stopped. The hub then answers <code>410 Gone</code> . Never print a cancelled badge.

2. Who does what

We give you	You own
A network cable into our router, and a port for it	A computer with a cable to that port, which runs your software all day
The print hub, and the address of the hub on the day	The listening for announcements, and the collection of badges
One announcement for every badge that a desk asks for	The printing: printers, stock, layout, drivers, ribbons and jams
The words of the badge, the QR code text, the design and a finished picture	The decision on how the badge looks, if you do not use our picture
A test hub, or a sample button, before the event	The message to us before the event that your system works

We want nothing back. Do not send a "printed" message. Do not send a "failed" message. Do not send printer status. If a badge does not come out, the desk shows that nobody collected the badge, and the desk tries again (section 8).

3. The steps in short

1. Put a network cable into our router. We give you the port.
2. **Use a cable. Do not use Wi-Fi.** Wi-Fi drops broadcasts, and Wi-Fi stops in a crowd. Badges must not wait for that. Use Wi-Fi only as a fallback, if a cable fails.
3. Listen for UDP datagrams on port **8632**. Each announcement tells you that a badge is ready, and where to collect the badge.
4. Make a `GET` request to that address. You get the full badge as JSON.
5. Also make a `GET /v1/prints` request every 5 seconds. Do this because you can miss an announcement.
6. Print the badge.
7. A badge that answers **410 Gone** is cancelled. **Never print it.**
8. Print each badge **one time**. Keep a record on disk of each `id` and each `jobKey` that you printed. Run one collector. Read section 8a.

4. Connecting

- **The network.** We give you a network cable and a port on the day. The hub and our iPads are on the same network. Your computer joins that network with the cable. We give you Wi-Fi details only as a fallback. Nothing goes over the internet. Datagrams do not cross routers. Therefore a different network does not work, and a guest network with "client isolation" does not work.
- **The address of the hub.** We tell you the address on the day. You can also find the address in two ways:
 - The hub is the sender of every announcement. The address is also in the `pickup` URL.
 - The hub advertises itself over Bonjour / mDNS as a `_fehub._tcp` service. Use `dns-sd -B _fehub._tcp` on a Mac, or use any Bonjour browser.
- **The ports.** HTTP is on TCP **8631**. Datagrams are on UDP **8632**. Your firewall must let UDP 8632 **in**, and TCP 8631 **out**.
 - **Windows:** Windows Defender Firewall asks you the first time that Python listens. Tick **Private networks**. Then choose **Allow access**. If you missed the question, open Windows Security > Firewall & network protection > Allow an app through firewall. Then tick Python for Private. Also set the event Wi-Fi to **Private** (Settings > Network & internet > Wi-Fi > the network > Private network).
 - **macOS:** macOS asks "Do you want the application Python to accept incoming network connections?" if the firewall is on. Choose **Allow**.
- **No login.** There is no key, no token and no password. The event network is the boundary. Only the devices that we let on the network can reach the hub. The routes below answer only while we keep the hub in print partner mode.

5. The announcement

For every badge, the hub sends one small JSON datagram (UTF-8) to UDP port **8632**. The hub sends the datagram as a broadcast to `255.255.255.255`, and also to the broadcast address of the local subnet (for example `192.168.8.255`). The hub sends the datagram **three times**, with a gap of about 200 ms. Therefore you usually hear the same announcement **six times or more**. You hear it more times if the hub is on more than one network. All of these datagrams carry the same `id`. Act on the first datagram, and ignore the others.

This is a real announcement (from `samples/doorbell.json`):

```
{"fehub":1,"type":"print","id":"31f3d165-c9f9-43e0-ae4e-83ce28e966c3","hub":"FE Partner Bench","pickup":"http://192.168.8.20:8631/v1/prints/31f3d165-c9f9-43e0-ae4e-83ce28e966c3","time":"2026-09-17T18:08:55.069692Z","test":true}
```

Field	Type	Meaning
fehub	number	Always 1. Ignore a datagram that does not have it.
type	string	Always "print". Ignore any other value.
id	string (UUID)	The job id of the badge. It is the same in every repeat.
hub	string	The name of the hub.
pickup	string (URL)	The address where you collect the badge.
time	string	The time when the desk asked, in UTC, in ISO 8601 (for example 2026-09-17T18:08:55.069692Z).
test	boolean	true for a test badge or a sample badge.

The announcement never contains a name, an organisation, a QR code or any other data about the person. That data comes only from the pickup URL.

Collect from the hub address that we gave you. Read the announcement as "check now". Then get `/v1/prints/<id>` from the address that we gave you. A hub can be on more than one network, therefore the host in `pickup` can be a host that you cannot reach. If a second hub is on the same network, the badges of that hub answer 404 on your hub, and you ignore them. If you have no address in your configuration, use the sender address of the announcement with the port and the path from `pickup`. The example client uses the address that you give it.

The network can lose an announcement. This is the reason why we ask you to use a cable. Always check the waiting list as well (section 7).

6. Collecting a badge

```
GET http://<hub>:8631/v1/prints/<id>
```

Send your own collector id with the request. Send the same value every time, and on every request. The hub then can tell your system from a second system on the network:

```
X-Print-Collector: acme-print-1
```

The value is any text that you want, up to 100 characters. Use a machine name, or make a UUID one time and keep it in a file. The example client makes a UUID and keeps it in `printed.json`.

The hub answers 200 OK with `Content-Type: application/json; charset=utf-8` and `Cache-Control: no-store`.

The first GET that succeeds marks the badge as collected, and the desk shows the badge as printed. **One badge goes to one collector.** A second request with the same X-Print-Collector returns the same badge. Therefore a lost reply is safe, and a retry is safe. A request with a different X-Print-Collector, or with no X-Print-Collector, gets 409 Conflict. That means that somebody else prints that badge. Read section 8a.

This is a real badge (from samples/pickup.json; the picture is short here):

```
{
  "id": "31f3d165-c9f9-43e0-ae4e-83ce28e966c3",
  "time": "2026-09-17T18:08:55.069673Z",
  "desk": "Bench page",
  "test": true,
  "badge": {
    "name": "Desk 1",
    "nameLocal": null,
    "organisation": "Sample badge – not a real event",
    "jobTitle": null,
    "type": "TEST",
    "qr": "FE1:D:CALIBRATION",
    "serial": null,
    "registrationId": null,
    "reprint": false,
    "reason": null
  },
  "event": null,
  "design": {
    "medium": "Label102x64",
    "showName": true,
    "showOrganisation": true,
    "showJobTitle": false,
    "showType": true,
    "showQr": true,
    "showLogo": false,
    "footerText": "bench sample",
    "textColor": null,
    "whiteAreaXMm": null,
    "whiteAreaYMm": null,
    "whiteAreaWidthMm": null,
    "whiteAreaHeightMm": null,
    "qrSizeMm": null,
    "layout": {
      "areaXMm": 3,
      "areaYMm": 3,
      "areaWidthMm": 96,
      "areaHeightMm": 58,
      "authored": false,
      "elements": [
        { "kind": "Name", "xMm": 2.5, "yMm": 2.5, "widthMm": 59.38, "heightMm": 9.7875, "sizeMm": 7.25, "bold": true, "align": "Left", "maxLines": 2 },
        { "kind": "NameLocal", "xMm": 2.5, "yMm": 12.2875, "widthMm": 59.38, "heightMm": 7.3514996, "sizeMm": 5.6549997, "bold": false, "align": "Left", "maxLines": 1 },
        { "kind": "Organisation", "xMm": 2.5, "yMm": 25.671, "widthMm": 59.38, "heightMm": 6.0319996, "sizeMm": 4.64, "bold": false, "align": "Left", "maxLines": 1 },
        { "kind": "BadgeType", "xMm": 2.5, "yMm": 49.004, "widthMm": 59.38, "heightMm": 6.496, "sizeMm": 4.64, "bold": true, "align": "Left", "maxLines": 1 },
        { "kind": "Qr", "xMm": 64.380005, "yMm": 14.440001, "widthMm": 29.119999, "heightMm": 29.119999, "sizeMm": 29.119999, "bold": false, "align": "Left", "maxLines": 1 }
      ]
    }
  }
}
```

```

    ]
  },
  "logo": null
},
"size": { "widthMm": 102, "heightMm": 64 },
"image": {
  "contentType": "image/png",
  "dpi": 203,
  "widthPx": 816,
  "heightPx": 512,
  "base64": "iVBORw0KGgoAAAANSUhEUgAAAzAAAAIA..."
},
"jobKey": "31f3d165-c9f9-43e0-ae4e-83ce28e966c3"
}

```

The badge of a real attendee has the same shape, with the fields filled in. For example, it has `"name": "Layla Example"`, `"nameLocal": "ليلى مثال"`, `"type": "Speaker"`, `"qr": "FE1:B:K7Q2M9"` and `"serial": "K7Q2M9"`. It also has `event` and `logo` set. `fake-partner-hub.py` serves badges of that shape.

Fields

The text is UTF-8 JSON. It can contain Arabic script or other scripts. Keep the text as UTF-8 when you save it. A field that this document shows as nullable can be `null`. A number can be a whole number (`102`), or it can have decimals (`7.3514996`). All lengths are millimetres. Treat an unknown extra field as normal. We can add a field later. We do not remove a field, and we do not rename a field, without a new spec version.

The top level

Field	Type	Meaning
<code>id</code>	string (UUID)	The job id of the badge. It is the same as the <code>id</code> in the announcement.
<code>time</code>	string	The time when the desk asked, in UTC, in ISO 8601. It can differ from the <code>time</code> in the announcement by one millisecond.
<code>desk</code>	string, nullable	The desk that asked, for example <code>"Desk 3"</code> . This is useful if you run more than one printer.
<code>test</code>	boolean	<code>true</code> for a test badge or a sample badge. Nobody wears that badge.
<code>badge</code>	object	The content of the badge (below).
<code>event</code>	object, nullable	The <code>id</code> (UUID) and the <code>name</code> of the event. It is <code>null</code> for a sample from a test hub.
<code>design</code>	object	The badge design of the event (below).
<code>size</code>	object	The <code>widthMm</code> and the <code>heightMm</code> of the badge.
<code>image</code>	object	The finished badge as a picture (below).
<code>jobKey</code>	string (UUID)	The id of the DESK for this badge. It stays the same across a desk retry, and also when the retry comes with a new <code>id</code> . Dedupe on this field — read section 8a.

badge : the hub sends every field, also when the design hides the field. You can then decide for yourself.

Field	Type	Meaning
<code>name</code>	string, nullable	The name of the person.
<code>nameLocal</code>	string, nullable	The name in the script of the person, for example Arabic (right-to-left).
<code>organisation</code>	string, nullable	The organisation of the person.
<code>jobTitle</code>	string, nullable	The job title of the person.
<code>type</code>	string, nullable	The attendee type label, for example <code>"Speaker"</code> , <code>"Delegate"</code> or <code>"Press"</code> . It is <code>"TEST"</code> on a sample.
<code>qr</code>	string, nullable	The exact text for the QR code. Encode this text, and nothing else. Our scanners read this text at the door.
<code>serial</code>	string, nullable	The serial of the badge (it is also inside <code>qr</code>). It is <code>null</code> on a sample.
<code>registrationId</code>	string (UUID), nullable	Our id for the registration of the person. It is <code>null</code> for a test and for a sample.
<code>reprint</code>	boolean	<code>true</code> if this badge replaces a badge that the person already had.
<code>reason</code>	string, nullable	The reason for the reprint, if the desk gave one. The hub sets it only when <code>reprint</code> is <code>true</code> .

design

Field	Type	Meaning
medium	string	The stock: <code>Label102x64</code> (a 102 × 64 mm label), <code>Label89x35</code> (an 89 × 35 mm label) or <code>Preprinted4x6</code> (a 4 × 6 in pre-printed card).
showName, showOrganisation, showJobTitle, showType, showQr, showLogo	boolean	The on/off setting of the event for each item. <code>layout</code> below shows what our picture contains.
footerText	string, nullable	Fixed text along the bottom, if the event has some.
textColor	string, nullable	The text colour as <code>#RRGGBB</code> . <code>null</code> means black.
whiteAreaXMm, whiteAreaYMm, whiteAreaWidthMm, whiteAreaHeightMm	number, nullable	On a pre-printed card: the blank window for the text, from the top-left corner of the card.
qrSizeMm	number, nullable	The size of the QR code, if the event set a size.
layout	object, nullable	The exact position of each item in our picture (below).
logo	object, nullable	The event logo: <code>contentType</code> (for example <code>image/png</code>) and <code>base64</code> .

design.layout

Field	Type	Meaning
areaXMm, areaYMm, areaWidthMm, areaHeightMm	number	The printable area, from the top-left corner of the badge.
authored	boolean	<code>true</code> if the designer of the event made the layout. <code>false</code> means our standard layout. In both cases, <code>elements</code> is what the hub drew.
elements	array	One entry for each item on the badge.

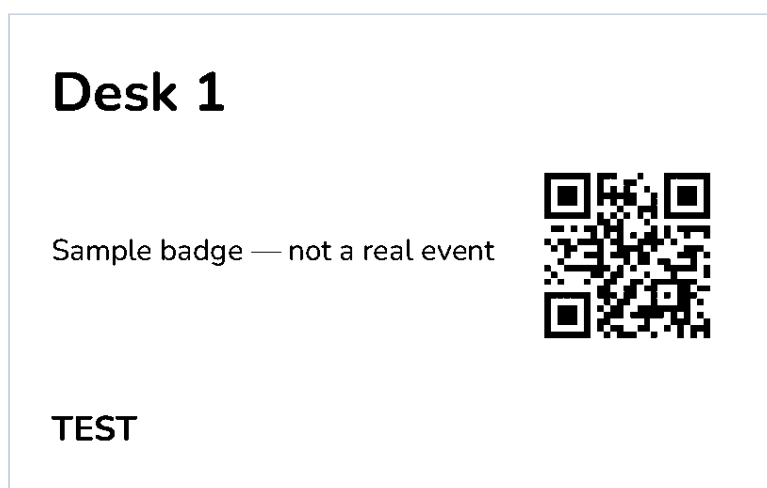
Each element has these fields:

Field	Type	Meaning
kind	string	Name , NameLocal , Organisation , JobTitle , BadgeType (the type label) or Qr .
xMm , yMm , widthMm , heightMm	number	The box of the item, from the top-left corner of the printable area (not of the badge).
sizeMm	number	The cap height of the text, before any shrink-to-fit. For Qr , it is the size of the square.
bold	boolean	Bold text.
align	string	Left , Centre or Right .
maxLines	number	The number of lines for the text. The hub then shrinks the text, and then cuts the text with "...".

size: widthMm and heightMm (numbers).

image: the finished badge, ready to print.

Field	Type	Meaning
contentType	string	Always image/png .
dpi	number	Dots per inch, usually 203. One pixel is one printer dot at this resolution.
widthPx , heightPx	number	The size of the picture. At 203 dpi, a 102 × 64 mm badge is 816 × 512.
base64	string	The PNG, in base64 (the standard alphabet, with padding).



The finished picture as the hub sends it: pure black on white, one pixel per printer dot.

The picture is pure black on white. The hub stores it as 8-bit RGB. It is exactly what our own thermal printers print. On a pre-printed card, the picture contains only the text and the QR code. It does not contain the artwork of the card. You have two choices. Print the picture at dpi with no scaling (widthPx / dpi inches wide). Or make the layout of the badge yourself from the fields above.

7. The waiting list

```
GET http://<hub>:8631/v1/prints
```

```
{
  "waiting": [
    {
      "id": "24342316-0fe8-46f9-b509-9c83aaf88cf6",
      "jobKey": "24342316-0fe8-46f9-b509-9c83aaf88cf6",
      "time": "2026-09-17T10:37:09.690477Z",
      "test": true,
      "pickup": "http://192.168.8.20:8631/v1/prints/24342316-0fe8-46f9-b509-9c83aaf88cf6"
    }
  ]
}
```

The waiting list gives every badge that nobody collected, with the oldest badge first. It does not give a collected badge, and it does not give a cancelled badge. The `pickup` field uses the same host that you called. Check the waiting list **every 5 seconds**. Collect each badge that you do not have. The waiting list is your safety net for a lost announcement, and for the moment when your software starts. A desk waits only 10 seconds. Therefore a longer gap between checks loses badges.

The waiting list also gives `jobKey`, as the badge does. Therefore you can skip a badge that you printed, **and you do not collect that badge at all**.

8. Timing, cancellation and repeats

- **The desk waits 10 seconds** for you to collect a badge. If you collect the badge in time, the desk shows the badge as printed. If you do not collect the badge, the desk shows "Not picked up by the print partner. Check their system is on this network."
- **The hub then cancels the badge.** From that moment, a request for the badge gets **410 Gone**. **Do not print that badge.** Do not ask for it again. The desk can already print a replacement, and two prints make two badges.
- A collection and a cancellation cannot both happen. If your `GET` arrives first, the badge is yours, and the desk shows that the badge printed. If the 10 seconds ended first, you get **410**.
- **The hub deletes a badge after 10 minutes.** For a collected badge, the 10 minutes start at the collection. For any other badge, the 10 minutes start when the hub made the badge. After that, the badge answers **404**.
- A restart of the hub cancels every badge that nobody collected.

When the desk asks for the same badge again, one of three things happens:

- You already collected the badge: the hub counts the badge as printed, and the hub announces nothing.
- The hub cancelled the badge: the hub announces the badge again with a **new id**. Collect that badge and print it as normal.
- The badge still waits: the hub announces the same `id` again.

Therefore one desk request never gives you two live job ids. There are other cases:

- **A reprint** (for a lost badge or a damaged badge) comes as a new badge with `"reprint": true`. It also has a `reason`, if the desk gave one. Print a reprint.
- **An event with a new badge each day**: the same person gets a new badge (a new `id` and a new `serial`) each day. Print each badge.
- **A test badge** (`"test": true`): print it to check your printer, or only collect it. Both are correct.

Keep a list of each `id` that you collected. The example client keeps this list. The list stops a repeat announcement from a second print. Section 8a gives all of the detail.

8a. Never print a badge twice

Never print a badge twice. A delegate with two badges is the complaint that we hear at the desk, and the badge with the older serial is the badge that stops at the door. Repeats are normal here. Repeats are not a fault. We announce each badge three times on every network, and you check the waiting list every 5 seconds. Therefore one badge reaches your software five or six times in the ordinary case. One print of that badge is your job as much as ours.

What we guarantee

- **One badge, one `id`**. Every announcement and every waiting-list entry for one badge carries the same `id`. The hub never uses that `id` for a different badge.
- **One collect**. The first `GET` of a badge collects the badge. A different collector that asks for the badge gets `409 Conflict`, and not the badge. Your own collector that asks again (with the same `X-Print-Collector`) gets the same badge. Therefore a lost reply never costs you a badge.
- **One live badge for each desk request**. A desk that asks twice, because it heard nothing back, does not make two badges. If you already collected the badge, the desk shows that the badge printed, and the hub announces nothing.
- **The `jobKey` survives a retry**. If the desk gave up after 10 s and asked again, the new offer has a **new `id`** and the **same `jobKey`**. The old `id` is already `410`, and you must never print it.
- **A cancelled badge stays cancelled**. `410` is final for that `id`.

What you must do

1. **Dedupe on id**. Ignore an announcement for an `id` that you already get, or that you already printed.
2. **Also dedupe on jobKey**. One `jobKey` is one badge, however many `ids` it comes under. The waiting list also carries `jobKey`. Therefore you can skip a badge that you printed, and you do not collect that badge at all.
3. **Keep that record on disk. Check the record at start-up**. A restart, a crash or a second copy of your software must not print the badges of this morning again. The example client keeps `printed.json` beside the badge folder. That file holds every `id` and every `jobKey` that the client finished. The client writes the record **before** it sends anything to a printer.
4. **Send X-Print-Collector**. Send the same value every time, and keep the value across a restart. This value lets us tell your retry from a second machine.
5. **Run one collector**. Use one computer, one process and one printer queue for each badge. A spare machine that runs all day, or a process from yesterday that nobody stopped, is the usual cause of two prints of everything. If you run a spare machine, give it its own `X-Print-Collector`. The hub then tells it `409` for each badge that the live collector took. That is the safety net. It is not the plan.
6. **Write the record before you print, and not after**. If your software stops between the collection and the print, the badge is not printed twice. The badge is simply not printed, and that is the correct case to fail on. Read the steps below.

The fields to key on

Field	Where	Use it for
<code>id</code>	announcement, waiting list, badge	This offer. Ignore each repeat of it.
<code>jobKey</code>	waiting list, badge	The badge of the desk. One <code>jobKey</code> is one printed badge, for ever.
<code>badge.serial</code>	badge	The serial in the QR code. It is useful in your own logs, and for the question "did we print this badge?" afterwards. It is <code>null</code> on a test badge and on a sample badge. Therefore never key on the serial alone.

What IS a new badge, and what you must print

- A **reprint** (`"reprint": true`, often with a `reason` such as "Lost badge"). It has a **new jobKey**, a new `id` and usually a new `serial`. The desk asked for it on purpose, because the delegate stands there with no badge. Never stop a reprint.
- A **new badge for a new day**, at an event that gives a badge each day: a new `jobKey` and a new `serial`. Print each badge.
- A **test badge** (`"test": true`): print it, or do not print it, as you want.

If a badge has a `jobKey` that you did not print, somebody waits for that badge. Print it.

If your printer jams after you collected the badge

We already count the badge as printed, and the desk has gone on. There is no way to give the badge back. Do this:

1. Clear the jam. Then print the badge from the copy that you saved. This is the reason why the example client writes the PNG file and the JSON file to disk before it prints.
2. If you cannot print the copy, tell the desk staff. They print the badge again from the iPad. That print has a **new jobKey**, and it is a new badge. The delegate then has one badge.

Never retry with a second collection. The `id` that you have is the only copy that we keep. A second collection of the badge of a different person is the duplicate that we all try to prevent.

How to test this

`fake-partner-hub.py` tries to make your client print twice. It announces everything three times. It offers one badge again under a new `id` with the same `jobKey` after you collect it ("Robin Retry"). It also keeps one badge on the waiting list for ever after the collection ("Ash Repeat"). Run two copies of your client at the same time. Then restart them. Then check that each badge came out exactly one time.

9. Status codes

An error is JSON (`application/problem+json`) with a plain-English `detail`:

```
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.11","title":"Print cancelled","status":410,"detail":"This print was cancelled because it wasn't collected in time."}
```

Status	Meaning	What to do
200	Here is the badge.	Print the badge (the collect route). Or collect each badge (the waiting list).
404	This hub has no badge with that id. The hub never made the badge, or the 10 minutes of the badge ended.	Drop the badge.
409	This code has two meanings. The <code>title</code> field tells them apart. "Not in print partner mode" : the hub prints on its own printers. "Already collected" : a different collector on this network has that badge.	Not in print partner mode: check every 5 s, and tell us if this state continues. Already collected: do not print the badge , and find the other collector (section 8a).
410	The hub cancelled the badge, because nobody collected it in time.	Do not print the badge . Drop it, and never ask for it again.
503	The hub is locked, because nobody started the desk today.	Check every 5 s.

If you cannot reach the hub at all, check that you are on the event network. Then retry with a longer delay each time. The example client waits 5 s, and then doubles the wait up to 60 s.

10. Your data duties

- You get only the data that we print on a badge, and only for a badge that a desk asked for. There is no attendee list, no search and no check-in data.
- Keep the badge data only for the time that you need it to print. Delete the saved badges (the PNG files and the JSON files of the example client) at the end of the event.
- Do not send the badge data anywhere else. Use it only to print.

11. Testing

On your own computer, with the stand-in hub. In one window, run:

```
python fake-partner-hub.py (Windows: py -3 fake-partner-hub.py)
```

In another window, run:

```
python print-partner-client.py --hub 127.0.0.1
```

The client saves three waiting badges immediately. It finds one of those three badges only through the waiting list. A fourth badge is cancelled, and the client must not save it. Open `http://127.0.0.1:8631/` and press **Send a sample print** to send more badges. That page tells you if your system collected the badge inside 10 seconds. There are three options. `--every 20` sends a sample badge every 20 seconds. `--mode manage` answers `409`. `--mode locked` answers `503`. Now stop your client, send a sample badge, wait 10 seconds, and start the client again. That badge must not print.

On our network, with our hub. Before the event, we can lend you a test hub, or we can meet you on site. Open `http://<hub>:8631/` in a browser. That page has these controls:

- **Use print partner mode** puts the hub on this interface.
- **Send a sample print** sends a sample badge, and tells you if your system collected it.
- The page shows the time of the last collection, and the number of badges that wait.

On the day, we change the mode from our app, and we can send you a test badge from our app.

12. Troubleshooting

Symptom	Likely cause
Nothing arrives, and you cannot reach the waiting list	You are not on the event network, or the hub address is wrong, or something blocks TCP 8631.
The waiting list works, but no announcement arrives	A firewall blocks UDP 8632 in (on Windows: the network is set to Public, or Python is not allowed). Or you are on Wi-Fi that isolates clients. Use the cable. Badges still arrive through the waiting list, but late.
"cannot listen on UDP 8632"	A different program holds the port, and does not share it. Close that program.
409 all the time, with "Not in print partner mode"	The hub is not in print partner mode. Ask us.
409 with "Already collected"	A different collector has that badge: a spare machine, an old process, or a second copy of your software. Stop the extra one. Do not print the badge.
A delegate has two badges	There are two collectors, or a client has no record of what it printed. Read section 8a.
503	Nobody started the desk. This clears when the staff start the desk.
The desk shows "Not picked up by the print partner"	Your system did not collect the badge inside 10 s. Check that your system runs, and that it is on the Wi-Fi.
410	The hub cancelled the badge. Your behaviour is correct: do not print it.
An Arabic name looks wrong in a saved file	Save the file as UTF-8. Your console can show the name wrongly, but the file is still correct.

13. What is not available

- There is no attendee list, no search and no check-in data.
- There is no way to send anything back. We do not need to know if you printed the badge, which printer you used, or what went wrong.
- There is no other part of the hub.
- There is nothing over the internet.

Changes

- 1 (17 Sep 2026): the first version. It gives the announcement, the pickup, the waiting list, every field and every status code, the rule that one badge is printed one time, and the test with the stand-in hub.

A. Getting started (the kit's README)

The developer kit ships this as `README.txt`. It is the short way in; the sections above are the reference.

FE PRINT PARTNER KIT

=====

This kit is for a print supplier. The system of that supplier prints the name badges at an FE event.

The model in one sentence:

We announce what we must print. You collect it and you print it.

At the event, our print hub sends a small UDP message on port 8632 each time that a check-in desk asks for a badge. This message is the announcement. Your computer collects the badge over HTTP from the hub on port 8631. There is no login and no key. You print the badge in the way that you want.

WHAT WE GIVE YOU	WHAT YOU BUILD
-----	-----
The event network and the hub	Your own printing, from the badge
The announcement on UDP 8632	picture or the badge data
The badge over HTTP on 8631:	Your queue, retries and operators
the words, the design, the QR	The printer, the stock and the
text and a finished picture	settings
This example client and the spec	Everything else at your end

Everything after the collection of a badge is yours. We send nothing else, and we want nothing back.

The full interface is in `print-partner-spec.md`: the announcement, the pickup, the waiting list, every field and every error code. That file is the reference. This file only starts you.

WORDS WE USE

announcement	The UDP message on port 8632 that tells you that a badge is ready. Section 1 of the spec calls this the announcement.
badge	The thing that you print, and also the data for it.
job id	The id of the hub for one offer of a badge ("id").
job key	The id of the desk for the badge ("jobKey").
collector	Your system, which collects badges.
collect	To get a badge from the hub over HTTP.
waiting list	The list of badges that nobody collected.
cancelled	The state of a badge that the hub stopped. Never print it.

REQUIREMENTS

Python 3.10 or a later version. You install nothing else.

macOS	Install from https://www.python.org/downloads/ , or with Homebrew: <code>brew install python</code>
Windows	Install from https://www.python.org/downloads/ and tick "Add python.exe to PATH" on the first screen. The "py" launcher that comes with it also works.

Check the version. Open Terminal (macOS) or Command Prompt (Windows). Then run

```
python3 --version      (macOS)
py -3 --version        (Windows)
```

QUICK START A – one computer, and no event

The kit contains a stand-in hub. It serves some badges with false data. Therefore you can develop without our hardware.

- macOS
1. Double-click `start-fake-hub-mac.command`. Leave the window open.
 2. Double-click `start-client-mac.command`. Press Enter to accept 127.0.0.1 as the hub address.

Windows

1. Double-click start-fake-hub-windows.bat. Leave the window open.
2. Double-click start-client-windows.bat. Press Enter to accept 127.0.0.1 as the hub address.

In a few seconds the client saves each badge into the "badges" folder next to these files. It saves <id>.png (the finished picture) and <id>.json (all of the badge data). The client finds one badge only through its regular check of the waiting list, therefore that badge can take about 10 seconds. The hub cancels one badge: the client skips that badge, and that is correct. Two more badges try to make the client print the same badge twice. The client does not print it twice, and your system must not print it twice. Read "NEVER PRINT A BADGE TWICE" below.

If macOS says that it cannot open the .command file, right-click the file and choose Open. Or run the file from Terminal: `bash start-client-mac.command`

Stop either window with Ctrl-C, or close the window.

QUICK START B – at the event, with the real hub

1. Put a network cable into our router. FE gives you the port. Use a cable. Do not use Wi-Fi: Wi-Fi drops broadcasts, and Wi-Fi stops in a crowd. We give you the Wi-Fi details only as a fallback.
2. FE gives you the address of the hub on the day (for example 192.168.8.20).
3. Start the client. Type that address when the client asks:
macOS double-click start-client-mac.command
Windows double-click start-client-windows.bat
Or give the address directly:
`bash start-client-mac.command 192.168.8.20`
`start-client-windows.bat 192.168.8.20`
4. Ask FE to send a test badge. A test badge has "test": true.

Do not run the stand-in hub at the event.

WHAT EACH FILE IS

README.txt	This file.
print-partner-spec.md	The full interface: the announcement, the pickup, the waiting list, the fields and the error codes.
print-partner-client.py	A working example client. Put your own printing in the function print_badge(). Everything else can stay as it is.
fake-partner-hub.py	The stand-in hub, with badges that hold false data. Use it only to develop.
start-client-mac.command	Starts the client (macOS).
start-fake-hub-mac.command	Starts the stand-in hub (macOS).
start-client-windows.bat	Starts the client (Windows).
start-fake-hub-windows.bat	Starts the stand-in hub (Windows).
samples/	Example badge data, if the kit contains it.

The launchers take optional arguments:

```
start-client-*  [hub address] [hub port]    (defaults 127.0.0.1, 8631)
start-fake-hub-* [port]                    (default 8631)
Windows only: add /nopause to close the window without a key press.
```

badges.printed.json The record of the client of what it printed.
Do not delete this file while the event runs.

To run the client directly:

```
python3 print-partner-client.py --hub <address> [--port 8631] [--out badges]
                                   [--record FILE] [--forget]
```

NEVER PRINT A BADGE TWICE

Never print a badge twice. A delegate with two badges is the complaint that we hear at the desk. The hub announces each badge three times, on every network, and the waiting list repeats each badge every 5 seconds. Therefore you hear about one badge five or six times in the ordinary case.

The hub gives each badge a job id ("id") and the job key of the desk ("jobKey"). The hub gives each badge to ONE collector, and it answers 409 to a second collector. The example client keeps every job id and every job key that it printed in `badges.printed.json`. It checks that file before it collects, and again before it prints. It sends its own X-Print-Collector on every request. Therefore a restart, a repeat announcement or a second client never prints a badge again. Copy that behaviour.

Run ONE collector. A spare machine, or an old process that nobody stopped, is the usual cause of two prints of everything.

A reprint, or a badge for a new day, comes with a NEW job key. It IS a new badge, and you must print it.

--forget empties the record, therefore you can print a test badge again. Never use --forget at an event.

Section 8a of `print-partner-spec.md` gives all of the detail.

FIREWALL

The client must receive UDP on port 8632. It must also connect out to TCP port 8631. Your firewall must allow both.

- | | |
|---------|--|
| macOS | If macOS asks "Do you want the application Python to accept incoming network connections?", choose Allow. If you missed the question: System Settings > Network > Firewall > Options, and set Python to "Allow incoming connections". |
| Windows | When Windows Defender Firewall asks about Python, tick "Private networks" and click Allow. If you missed the question: Control Panel > Windows Defender Firewall > Allow an app through Windows Defender Firewall, and tick Private for Python. Also set the event network as a Private network. |

RULES THAT MATTER

- A badge that answers 410 is cancelled. Do not print it. The desk stopped the wait, and the desk can already print another badge.
- The hub can announce the same badge more than one time. Print it one time.
- We do not need a "printed" reply or a "failed" reply. Send nothing back to the hub.
- A badge holds personal data. Delete the saved files after the event.

TROUBLESHOOTING

No badges arrive:

- Is your cable in our router, and is the link light on?
- Does the firewall block Python? Read FIREWALL above.
- Start the client with the address of the hub (--hub <address>, or type the address when the client asks). The client then also checks the waiting list of the hub every 5 seconds. That works also when the network blocks the announcement.
- "cannot reach the hub": check the address with FE. Also check that you can open `http://<address>:8631/v1/prints` in a browser.
- "hub not ready (HTTP 409)": the hub is not yet in print partner mode. Ask FE to switch the mode on.
- "hub not ready (HTTP 503)": nobody started the desk. Wait, or ask FE.

"Address already in use" with the stand-in hub:

A second copy already runs. Close that copy, or start with another port:

`start-fake-hub-mac.command 8641` and then `start-client-mac.command`
`127.0.0.1 8641` (the same on Windows).

Only one program on a computer can listen for the announcement at one time. If a different listener runs, the client still finds badges through the waiting list.

</content>